



COMPUTAÇÃO EM NUVEM

AULA 3



Prof. Maike Cristian Rebelo de Lima



CONVERSA INICIAL

Nesta etapa, vamos explorar como a computação em nuvem está transformando a forma como as empresas e organizações consomem recursos de TI, oferecendo escalabilidade, flexibilidade e eficiência sem precedentes.

Vamos nos aprofundar no mundo da computação em nuvem e conhecer como ela é fundamental no cenário atual de TI. Vamos estudar os conceitos básicos de computação em nuvem, os diferentes modelos de serviço, as arquiteturas mais comuns e os principais serviços oferecidos pelos principais provedores de nuvem, como a Amazon Web Services (AWS), Microsoft Azure e Google Cloud Platform (GCP). Para isso, abordaremos os seguintes tópicos:

1. Serviços de computação em nuvem:

- Definição e importância dos serviços de computação em nuvem;
- Visão geral dos principais provedores: AWS, Azure e GCP.

2. Máquinas virtuais (VMs):

- Conceito e importância das máquinas virtuais;
- Introdução ao Amazon EC2, Azure Virtual Machines e Google Compute Engine;
- Comparação entre as VMs dos três provedores.

3. Provisionamento e dimensionamento de recursos:

- Conceitos de provisionamento e dimensionamento;
- Diferenciação entre dimensionamento vertical e horizontal;
- Serviços de Auto Scaling: AWS Auto Scaling, Azure Scale Sets e Google Cloud Autoscaler.

4. Gerenciamento de máquinas virtuais e contêineres:

- Conceitos de automação e orquestração de contêineres;
- Serviços de orquestração: Amazon ECS, Amazon EKS, Azure Kubernetes Service (AKS) e Google Kubernetes Engine (GKE).

5. Processamento em lote:

- Introdução ao processamento em lote;
- Serviços de processamento em lote na AWS, Azure e GCP: AWS Batch, Azure Batch e Google Cloud Dataflow.

Ao final desta etapa, esperamos que você tenha uma compreensão sólida de como a computação em nuvem funciona, incluindo os principais serviços e



ferramentas oferecidos pelos três maiores provedores de nuvem, e estará preparado para explorar e trabalhar com essas tecnologias em seus projetos futuros.

TEMA 1 – SERVIÇOS DE COMPUTAÇÃO

Conforme tratamos anteriormente, serviços de computação em nuvem são produtos oferecidos pelos provedores de *cloud* visando a uma abstração de algumas etapas que necessitavam ser realizadas, para se ter o provisionamento desses serviços. Dessa forma, iremos realizar uma recapitulação dos serviços, seus benefícios e os modelos de serviços dos provedores, para que possamos avançar.

Podemos conceituar computação em nuvem como a entrega de diferentes serviços pela internet. Esses serviços incluem armazenamento de dados, banco de dados, redes, software, entre outros. Esse novo modelo de entrega de serviços de computação permite que empresas acessem e utilizem esses recursos de acordo com as suas necessidades, pagando apenas pelo que utilizam. Isso elimina a necessidade de investimentos iniciais pesados em infraestrutura de Tecnologia da Informação (TI) e permite uma escalabilidade rápida e eficiente.

Podemos considerar a abstração de etapas que eram demoradas, como provisionamento de servidores em *datacenter*, acoplamento de *storage*, comunicação desses servidores em redes e sua entrega para a equipe responsável pelo servidor. Esse processo demorava meses e necessitava de um número grande de pessoas envolvidas, como Gerente de Projetos focado em entregas de Infraestrutura, pessoal técnico para cada atividade, além do tempo que o serviço demorava para estar *up and running* e entregue para as áreas responsáveis.

1.1 Modelos de serviço em nuvem

Vamos explorar três principais modelos de serviço em nuvem:

1. **Infrastructure as a Service (IaaS):** é o nível mais básico de serviço em nuvem, onde você aluga infraestrutura de TI – servidores e máquinas virtuais, armazenamento, redes e sistemas operacionais – de um



provedor de nuvem em uma base de pagamento conforme o uso.
Ex.: Amazon EC2, Azure Virtual Machine e Google Compute Engine.

2. **Platform as a Service (PaaS):** esse modelo fornece um ambiente de desenvolvimento e implantação completo na nuvem, com recursos que permitem entregar tudo, desde aplicativos simples baseados na nuvem até sofisticados aplicativos empresariais prontos para a nuvem.
Ex.: AWS Elastic Beanstalk, Azure App Services e Google App Engine.
3. **Software as a Service (SaaS):** nesse modelo, você usa um software baseado na web hospedado na nuvem. As aplicações SaaS estão disponíveis por meio de assinatura e são acessadas via internet.
Ex.: Microsoft Office 365, Google Workspace.

1.2 Benefícios da computação em nuvem

Os benefícios que podemos listar da computação em nuvem, são:

- **Escalabilidade:** você pode aumentar ou diminuir os recursos e serviços de acordo com a demanda.
- **Flexibilidade:** a capacidade de acessar recursos de TI de qualquer lugar, a qualquer hora.
- **Custo-Benefício:** redução de custos operacionais e eliminação de gastos com hardware.
- **Alta Disponibilidade:** infraestrutura robusta e redundante que garante a disponibilidade dos serviços.
- **Segurança:** provedores de nuvem investem fortemente em medidas de segurança para proteger os dados dos clientes.

1.3 Visão geral dos provedores de nuvem

1. Amazon Web Services (AWS):

- Fundada em 2006, AWS é a líder de mercado em serviços de computação em nuvem.
- Oferece uma ampla gama de serviços, incluindo computação, armazenamento, bancos de dados, aprendizado de máquina, entre outros.



2. Microsoft Azure:

- Lançado em 2010, o Azure é conhecido por sua forte integração com produtos Microsoft.
- Oferece serviços semelhantes ao AWS, com ênfase em ambientes híbridos e soluções empresariais.

3. Google Cloud Platform (GCP):

- Conhecido por sua inovação em serviços e produtos de *big data* e aprendizado de máquina.
- Oferece serviços de computação, armazenamento e ferramentas de desenvolvimento robustas.

TEMA 2 – MÁQUINAS VIRTUAIS

Para introduzir o tema de máquinas virtuais precisamos passar pelo conceito de virtualização. A virtualização é a criação de uma versão virtual de um recurso de TI, como um servidor, uma estação de trabalho, um dispositivo de armazenamento ou uma rede. Essa tecnologia permite que vários sistemas operacionais e aplicativos sejam executados em um único servidor físico.

Uma máquina virtual (VM) é um ambiente computacional que emula um sistema de computador físico. Ela permite que um único hardware físico execute múltiplos sistemas operacionais simultaneamente. As VMs são criadas e gerenciadas por um software chamado hipervisor, que pode ser do tipo 1 (*bare-metal*) ou do tipo 2 (*hosted*). As VMs funcionam pela virtualização de hardware. O hipervisor intercepta as instruções do sistema operacional convidado (guest OS) e as traduz para o hardware físico subjacente. Isso permite que múltiplos sistemas operacionais compartilhem os mesmos recursos físicos (CPU, memória, armazenamento) sem interferir uns com os outros. Cada VM possui seu próprio sistema operacional, aplicativos e configurações, funcionando de forma isolada das outras VMs no mesmo *host*.

2.1 Principais conceitos de virtualização

O **hipervisor** (*hypervisor*) é o software responsável por criar e gerenciar máquinas virtuais (VMs). Existem dois tipos principais:

- Hipervisor tipo 1 (*bare-metal*): Executado diretamente no hardware físico. Ex.: VMware, ESXi, Microsoft Hyper-V.



- Hipervisor tipo 2 (*hosted*): Executado sobre um sistema operacional e aplicativo, rodando de forma isolada umas das outras, mas compartilhando os recursos físicos do host.

2.2 História e evolução das VMs

As máquinas virtuais têm origem nas décadas de 1960 e 1970, quando a IBM desenvolveu os primeiros sistemas de tempo compartilhado. Nos anos 1990, com o advento da computação pessoal, a VMware introduziu as VMs para desktops, permitindo que usuários comuns pudessem rodar múltiplos sistemas operacionais em um único PC. Com o crescimento da computação em nuvem nos anos 2000, a tecnologia de virtualização se tornou fundamental para os data centers, permitindo a criação de ambientes de nuvem altamente escaláveis e flexíveis. Hoje, as VMs são usadas em uma ampla variedade de aplicações, desde desenvolvimento de software até ambientes de produção em larga escala.

2.3 Principais casos de uso das VMs

1. **Desenvolvimento e teste de software:** Desenvolvedores podem criar e testar aplicativos em diferentes sistemas operacionais sem precisar de hardware adicional.
2. **Consolidação de servidores:** Permite que várias aplicações e serviços sejam executados em um único servidor físico, otimizando o uso dos recursos.
3. **Ambientes de produção:** Empresas utilizam VMs para hospedar aplicações de produção, aproveitando a escalabilidade e flexibilidade da virtualização.
4. **Recuperação de desastres:** Facilita a criação de planos de recuperação de desastres, pois as VMs podem ser facilmente replicadas e restauradas em diferentes locais.
5. **Educação e treinamento:** Instituições educacionais utilizam VMs para ensinar habilidades de TI, permitindo que os alunos experimentem diferentes sistemas operacionais e configurações de rede sem risco.



2.4 Vantagens e desvantagens das VMs

As VMs são um componente essencial da infraestrutura de TI, proporcionando uma série de benefícios que transformaram a forma como gerenciamos e utilizamos recursos computacionais. No entanto, como qualquer tecnologia, elas também apresentam desafios e limitações. Vamos explorar em detalhes as principais vantagens e desvantagens das VMs.

Tabela 1 – Vantagens do uso de VMs

Vantagem	Descrição
Isolamento	Cada VM opera de maneira independente, aumentando a segurança e a estabilidade. Problemas em uma VM não afetam as outras.
Flexibilidade	Permitem a execução de diferentes sistemas operacionais no mesmo hardware, facilitando diversos usos e testes.
Escalabilidade	Facilitam o dimensionamento dos recursos de acordo com a demanda, podendo adicionar ou remover capacidade conforme necessário.
Custo-benefício	Reduzem a necessidade de hardware físico adicional, economizando em custos de infraestrutura e manutenção.
Gerenciamento simplificado	Facilitam a administração e manutenção de sistemas, incluindo backups e migrações, tornando a gestão mais eficiente.

Tabela 2 – Desvantagens do uso de VMs

Desvantagem	Descrição
Desempenho	Pode haver uma sobrecarga de desempenho devido à camada de virtualização, resultando em menor eficiência comparada ao hardware dedicado.
Complexidade	A gestão de múltiplas VMs pode ser complexa, exigindo ferramentas e habilidades especializadas para administrar o ambiente virtual.

TEMA 3 – PROVISIONAMENTO E DIMENSIONAMENTO

Provisionamento é o processo de configurar e disponibilizar recursos de TI, como servidores, armazenamento e redes, conforme a necessidade. No contexto de computação em nuvem, o provisionamento pode ser dinâmico, ajustando-se automaticamente às demandas, ou estático, configurado manualmente.

O dimensionamento se refere ao ajuste dos recursos computacionais alocados para uma aplicação ou serviço, podendo ser feito de forma vertical



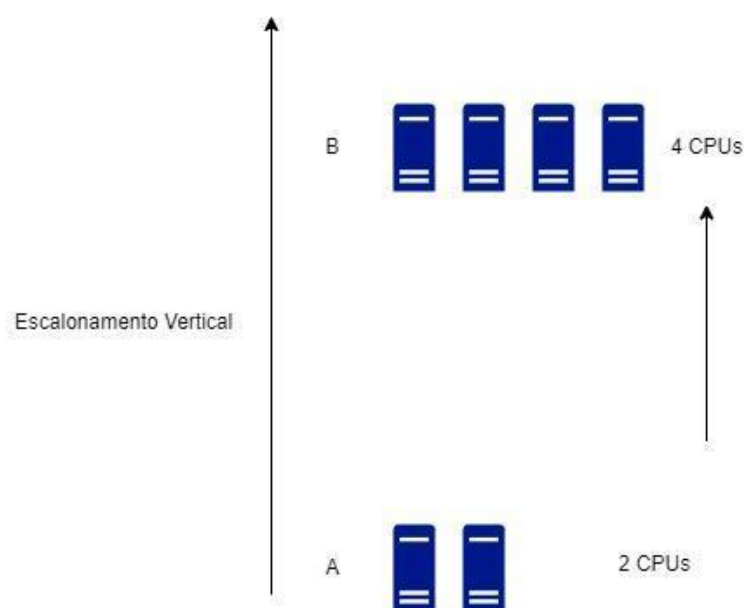
(*scale-up*) ou horizontal (*scale-out*). Esse ajuste é essencial para garantir que a aplicação tenha os recursos necessários para operar eficientemente, especialmente em cenários de variação de carga.

O provisionamento e dimensionamento adequados são cruciais para garantir eficiência operacional, otimização de custos e capacidade de resposta às mudanças na demanda em tempo real. Em ambientes de nuvem, essas práticas permitem uma utilização mais eficiente dos recursos, melhorando a performance e a resiliência dos sistemas.

3.1 Conceito de dimensionamento vertical (*scale-up*)

Quando um recurso de TI é substituído por outro com capacidade maior ou menor, é considerado dimensionamento vertical (Figura 1). Especificamente, quando a substituição do recurso é feita para aumento de capacidade, ela é considerada aumento de escala (*scale up*) e, quando o recurso de tecnologia é realizado para redução de capacidade, é tomado como diminuição de escala (*scale down*). O escalonamento vertical é menos comum em ambientes de nuvem devido ao tempo de inatividade necessário enquanto a substituição está a ocorrer.

Figura 1 – Um recurso de TI (um servidor virtual com duas CPUs) é ampliado, substituindo-o por um recurso informático mais potente com maior capacidade de armazenamento de dados (um servidor físico com quatro CPUs)





3.1.1 Como funciona o dimensionamento vertical

O dimensionamento vertical, também conhecido como *scale-up*, envolve a adição de mais recursos a uma máquina virtual ou servidor existente. Isso pode incluir aumentar a quantidade de CPU, memória (RAM), armazenamento ou outras capacidades de hardware. O objetivo é melhorar o desempenho e a capacidade da máquina virtual sem alterar a arquitetura da aplicação.

No contexto da nuvem, o dimensionamento vertical é geralmente realizado por meio do painel de controle do provedor de serviços de nuvem. Por exemplo, se uma aplicação em uma instância EC2 da AWS precisa de mais memória, o administrador pode simplesmente escolher uma instância de maior capacidade dentro do painel de controle da AWS. Após a modificação, a instância existente será reinicializada com os recursos aumentados, proporcionando mais poder computacional ou armazenamento para a aplicação.

Esse processo é relativamente simples e direto, sendo uma escolha popular quando a aplicação não é projetada para escalar horizontalmente, ou quando a carga de trabalho é monolítica e não se beneficia da distribuição em várias máquinas.

3.1.2 Vantagens do dimensionamento vertical

1. **Simplicidade de implementação:** O dimensionamento vertical é geralmente mais simples de implementar em comparação com o dimensionamento horizontal. Ele não requer alterações significativas na arquitetura da aplicação ou na infraestrutura subjacente. Os administradores podem simplesmente adicionar mais recursos a uma instância existente pelo painel de controle do provedor de nuvem.
2. **Consistência e integridade de dados:** Como o dimensionamento vertical mantém a aplicação em uma única instância, ele evita os problemas de consistência e integridade de dados que podem surgir ao distribuir a carga de trabalho em várias instâncias. Isso é particularmente benéfico para aplicações que não são projetadas para operar em um ambiente distribuído.
3. **Menor latência interna:** Adicionar recursos a uma única instância pode resultar em menor latência interna em comparação com a comunicação entre várias instâncias. Isso é vantajoso para aplicações que têm



operações intensivas de leitura e escrita que ocorrem dentro de uma única base de dados ou memória.

4. **Gerenciamento simplificado:** Gerenciar uma única instância com mais recursos pode ser mais simples do que gerenciar várias instâncias menores. Isso inclui tarefas como monitoramento, manutenção e atualização de software, que podem ser realizadas de maneira mais centralizada.

3.1.3 Desvantagens do dimensionamento vertical

1. **Limitações físicas do hardware:** O dimensionamento vertical é limitado pelas capacidades máximas do hardware subjacente. Cada máquina física tem um limite em termos de CPU, memória e armazenamento que pode ser alocado a uma instância. Uma vez que esses limites são atingidos, não é possível escalar verticalmente além disso.
2. **Ponto único de falha:** Uma aplicação dimensionada verticalmente depende de uma única instância. Se essa instância falhar, toda a aplicação pode ficar indisponível. Isso representa um risco significativo para a continuidade do serviço, especialmente em aplicações críticas.
3. **Custo:** Instâncias com mais recursos são geralmente mais caras do que múltiplas instâncias menores. Isso pode resultar em custos mais altos de operação, especialmente se a aplicação não estiver utilizando de forma eficiente todos os recursos adicionais.
4. **Downtime para atualizações:** Aumentar os recursos de uma instância existente geralmente requer reinicialização, resultando em *downtime*. Durante esse período, a aplicação pode ficar indisponível, o que pode ser problemático para serviços que precisam de alta disponibilidade.

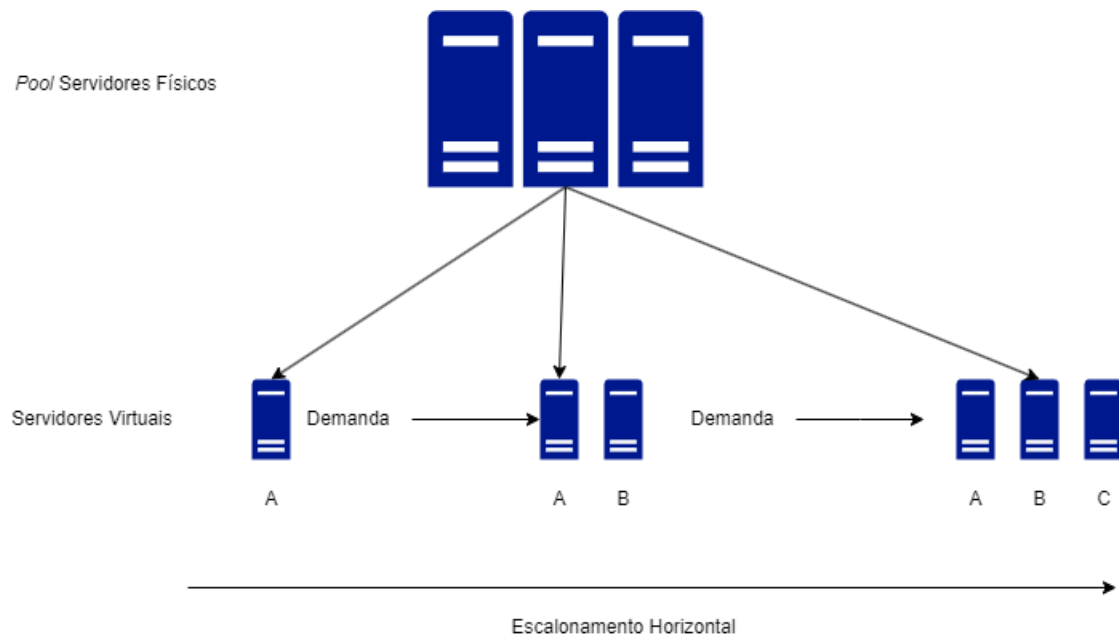
3.2 Conceito de dimensionamento horizontal (*scale-out*)

O dimensionamento horizontal, também conhecido como *scale-out*, envolve a adição de mais máquinas virtuais ou servidores para distribuir a carga de trabalho (Figura 2). Em vez de aumentar os recursos de uma única instância, o dimensionamento horizontal permite que múltiplas instâncias trabalhem juntas para gerenciar o aumento da demanda. Essa abordagem é particularmente



eficaz para aplicações que podem ser facilmente distribuídas e que requerem alta disponibilidade e resiliência.

Figura 2 – Um recurso de TI (Virtual Server A) é escalonado adicionando mais recurso de TI (Virtual Servers B e C)



3.2.1 Como funciona o dimensionamento horizontal

O dimensionamento horizontal funciona adicionando mais instâncias de máquinas virtuais ou servidores para dividir a carga de trabalho. Em um ambiente de nuvem, isso é geralmente gerenciado por meio de serviços de auto *scaling* fornecidos pelo provedor de nuvem. Por exemplo, em um serviço web, quando o tráfego aumenta, o sistema automaticamente cria novas instâncias para lidar com o aumento de carga.

A comunicação entre as instâncias é gerenciada por balanceadores de carga que distribuem as solicitações de forma equitativa entre todas as instâncias disponíveis. Cada instância opera de maneira independente, mas elas colaboram para fornecer uma única interface de serviço ao usuário final. Quando a demanda diminui, o sistema pode automaticamente desligar algumas instâncias para economizar recursos e reduzir custos.



3.2.2 Vantagens do dimensionamento horizontal

1. **Alta disponibilidade:** O dimensionamento horizontal aumenta a disponibilidade da aplicação, pois a carga de trabalho é distribuída entre várias instâncias. Se uma instância falhar, outras instâncias podem continuar a operar, garantindo a continuidade do serviço.
2. **Escalabilidade ilimitada:** Teoricamente, o dimensionamento horizontal oferece escalabilidade ilimitada, já que é possível adicionar tantas instâncias quanto necessário para atender à demanda. Isso é especialmente útil para aplicações com variações de carga imprevisíveis.
3. **Melhor distribuição de carga:** Com a ajuda de balanceadores de carga, o dimensionamento horizontal distribui a carga de trabalho de forma equitativa entre todas as instâncias, evitando sobrecargas em uma única instância.
4. **Redundância:** A presença de múltiplas instâncias fornece redundância, aumentando a resiliência do sistema. Em caso de falha de hardware ou software em uma instância, outras instâncias podem assumir a carga de trabalho.

3.2.3 Desvantagens do dimensionamento horizontal

1. **Complexidade de gerenciamento:** Gerenciar um ambiente com múltiplas instâncias pode ser mais complexo do que gerenciar uma única instância com mais recursos. Isso inclui a configuração, monitoramento e manutenção de várias máquinas.
2. **Custo Inicial de configuração:** Implementar dimensionamento horizontal pode exigir um investimento inicial maior em termos de configuração de infraestrutura, balanceamento de carga e arquitetura de aplicação.
3. **Consistência de dados:** Garantir a consistência dos dados em um ambiente distribuído pode ser desafiador. Problemas como a sincronização de dados e a latência de rede precisam ser cuidadosamente gerenciados.
4. **Requisitos de arquitetura de aplicação:** Nem todas as aplicações são adequadas para dimensionamento horizontal. Aplicações monolíticas ou



que dependam de um estado compartilhado podem precisar de modificações significativas para suportar uma arquitetura distribuída.

3.3 Comparação entre dimensionamento vertical e horizontal

Tabela 3 – Comparação entre escalonamento vertical e horizontal

Escalonamento horizontal	Escalonamento vertical
Menos dispendioso (por meio de componentes de hardware de base).	Mais dispendioso (servidores especializados).
Recursos de tecnologia instantaneamente disponíveis.	Recursos de tecnologia normalmente estão instantaneamente disponíveis.
Réplica de recursos e escalonamento automático.	Inicialização adicional normalmente é necessário.
Necessários recursos adicionais de tecnologia.	Não são necessários recursos adicionais.
Sem limite de capacidade de <i>hardware</i> .	Limitado à capacidade máxima de <i>hardware</i> .

TEMA 4 – GERENCIAMENTO DE MÁQUINAS VIRTUAIS E CONTÊINERES

4.1 Introdução ao gerenciamento de máquinas virtuais e contêineres

O gerenciamento de máquinas virtuais (VMs) e contêineres é uma prática essencial na administração de infraestruturas de TI modernas. As máquinas virtuais permitem a execução de múltiplos sistemas operacionais em um único hardware físico, proporcionando isolamento, flexibilidade e otimização de recursos. Contêineres, por outro lado, são uma forma mais leve de virtualização que encapsula uma aplicação e suas dependências em um pacote executável, facilitando a portabilidade e a consistência entre diferentes ambientes de desenvolvimento e produção.

Sua importância está no gerenciamento eficaz de VMs e contêineres, crucial para garantir a eficiência operacional, a escalabilidade e a resiliência das aplicações. Ele permite que as organizações respondam rapidamente às mudanças na demanda, implementem novas aplicações com agilidade e mantenham um alto nível de disponibilidade e segurança. Ferramentas de automação e orquestração são vitais nesse gerenciamento, simplificando a configuração, o monitoramento e a manutenção dos recursos.



4.1.1 História e evolução

- **História:** O conceito de máquinas virtuais remonta aos *mainframes* da IBM nos anos 1960, quando a necessidade de maximizar o uso do caro tempo de computação levou ao desenvolvimento da virtualização. No final dos anos 1990 e início dos anos 2000, a VMware popularizou a virtualização em servidores x86, tornando-a uma tecnologia *mainstream*.
- **Evolução:** Com a evolução da tecnologia, a virtualização se tornou mais eficiente e acessível, culminando na adoção generalizada em data centers e ambientes de nuvem. Nos últimos anos, os contêineres emergiram como uma tecnologia complementar, com o Docker, em 2013, liderando essa revolução. Contêineres oferecem uma forma de virtualização em nível de sistema operacional, permitindo uma utilização ainda mais eficiente dos recursos e facilitando o desenvolvimento e a implantação contínua.
- **Marcos históricos:**
 - 1960s: Início da virtualização em *mainframes*.
 - 1999: Fundação da VMware, popularizando a virtualização em servidores x86.
 - 2006: Lançamento do Amazon EC2, introduzindo a virtualização de servidores na nuvem pública.
 - 2013: Lançamento do Docker, revolucionando a tecnologia de contêineres.
 - 2014: Google lança o Kubernetes, uma plataforma de orquestração de contêineres, que rapidamente se torna a líder de mercado.

4.1.2 Principais desafios no gerenciamento

Gerenciar VMs e contêineres apresenta vários desafios que precisam ser enfrentados para garantir um ambiente de TI eficiente e seguro.

1. **Complexidade:** A administração de um grande número de VMs e contêineres pode ser complexa, exigindo ferramentas avançadas de automação e orquestração para simplificar o gerenciamento. A complexidade aumenta com a necessidade de integrar diferentes tipos de infraestruturas e plataformas de nuvem.



2. **Escalabilidade:** Manter a escalabilidade eficiente é um desafio constante. À medida que a demanda cresce, o sistema deve ser capaz de adicionar e remover recursos dinamicamente sem causar interrupções nos serviços.
3. **Segurança:** Garantir a segurança em um ambiente virtualizado e de contêineres é crucial. Isso inclui a proteção contra vulnerabilidades, a implementação de políticas de acesso rigorosas e a garantia de que os dados estão seguros durante a transmissão e o armazenamento.
4. **Monitoramento e diagnóstico:** Monitorar a saúde e o desempenho de VMs e contêineres em tempo real é essencial para identificar e resolver problemas rapidamente. Ferramentas de monitoramento avançadas são necessárias para fornecer visibilidade completa sobre o estado dos recursos.
5. **Interoperabilidade:** Integrar VMs e contêineres com outros sistemas e serviços pode ser desafiador, especialmente em ambientes de múltiplas nuvens ou híbridos. A interoperabilidade é fundamental para garantir que diferentes componentes do sistema funcionem harmoniosamente.
6. **Automação:** Automatizar processos de gerenciamento é vital para reduzir a carga de trabalho manual e minimizar erros humanos. A implementação de pipelines de CI/CD e a utilização de scripts de automação podem ajudar a atingir esses objetivos.

4.2. Ferramentas de orquestração

4.2.1 Kubernetes

4.2.1.1 Conceito e arquitetura

Kubernetes, muitas vezes abreviado como K8s, é uma plataforma de orquestração de contêineres *open source* originalmente desenvolvida pela Google e agora mantida pela Cloud Native Computing Foundation (CNCF). Kubernetes facilita a implantação, o dimensionamento e a gestão de aplicações containerizadas, permitindo que elas sejam executadas de maneira eficiente e resiliente em ambientes de produção.

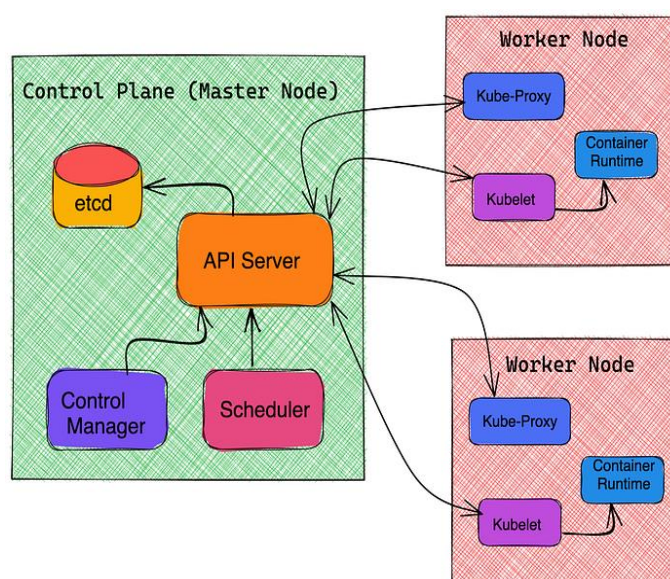
4.2.1.2 Arquitetura

Kubernetes adota uma arquitetura de mestre-escravo composta por vários componentes principais:

- Master Node: Coordena o *cluster*, gerenciando a carga de trabalho e a comunicação entre os diferentes componentes.
- API Server: Interface principal para interação com o *cluster*.
- Scheduler: Atribui contêineres aos nós com base nos requisitos de recursos e políticas de escalabilidade.
- *Controller manager*: Garante que o estado desejado do *cluster* seja mantido, gerenciando controladores como ReplicaSets e Deployments.
- etcd: Armazena dados de configuração e estado do *cluster* de forma distribuída.
- *Worker nodes*: Executam os contêineres e gerenciam a execução das aplicações.
- Kubelet: Agente que garante que os contêineres estejam em execução conforme especificado nas definições de *pods*.
- Kube-proxy: Gerencia as regras de rede e balanceamento de carga.
- *Container runtime*: Software que executa os contêineres, como Docker ou Containerd.

A Figura 3 ilustra a disposição e comunicação entre os componentes, como descrito acima.

Figura 3 – Componentes internos do Kubernetes





4.2.1.2 Componentes principais (*Pods*, *nodes*, *clusters*)

- **Pods:** Unidade básica de execução no Kubernetes, um *pod* pode conter um ou mais contêineres que compartilham o mesmo *namespace* de rede e volumes de armazenamento.
- **Nodes:** Máquinas, físicas ou virtuais, que compõem o *cluster* do Kubernetes. Cada nó contém os componentes necessários para executar *Pods* e é gerenciado pelo plano de controle do Kubernetes.
- **Clusters:** Conjunto de nós que trabalham juntos para executar aplicações containerizadas. O *cluster* proporciona uma única unidade de computação que permite o gerenciamento centralizado de recursos e *workloads*.

4.2.1.3 Exemplos de uso

- **Hospedagem de microservices:** O Kubernetes facilita o gerenciamento e a escalabilidade de *microservices*, permitindo implantações contínuas e fácil recuperação de falhas.
- **Plataformas de *big data*:** Utilizado para orquestrar *jobs* de processamento de dados em larga escala, como pipelines de ETL.
- **Aplicações web:** O Kubernetes automatiza o balanceamento de carga e o escalonamento de servidores web, garantindo alta disponibilidade e desempenho.

4.2.1.4 Vantagens e desvantagens

- **Vantagens:**
 - **Escalabilidade automática:** Ajusta automaticamente os recursos com base na demanda.
 - **Alta disponibilidade:** Garante a continuidade dos serviços mesmo em caso de falhas nos nós.
 - **Portabilidade:** Facilita a movimentação de aplicações entre diferentes ambientes (desenvolvimento, teste, produção).
- **Desvantagens:**
 - **Complexidade:** Requer um conhecimento aprofundado para configurar e gerenciar eficientemente.
 - **Curva de aprendizado:** Pode ser desafiador para equipes que não estão familiarizadas com a orquestração de contêineres.



- **Recursos necessários:** Demanda recursos significativos para operar em escala, tanto em termos de hardware quanto de habilidades técnicas.

4.2.2 Distribuições leves de Kubernetes

- **Conceito e arquitetura:** Distribuições leves de Kubernetes são versões otimizadas e simplificadas deste recurso, projetadas para ambientes com recursos limitados, como dispositivos IoT, bordas de rede e desenvolvedores que precisam de uma configuração rápida e simples para testes locais.
- **Arquitetura:** Essas distribuições mantêm os componentes essenciais do Kubernetes, mas com otimizações que reduzem a complexidade e o consumo de recursos. Alguns exemplos populares são:
 - **k3s:** Desenvolvido pela Rancher Labs, k3s é uma versão compacta de Kubernetes que requer menos memória e CPU, tornando-o ideal para ambientes de borda e desenvolvimento local.
 - **MicroK8s:** Mantido pela Canonical, o MicroK8s oferece uma instalação de Kubernetes fácil e rápida, com baixo *overhead*, ideal para desktops e servidores leves.
 - **Minikube:** Um projeto que facilita a execução de um *cluster* Kubernetes local, focado em ambientes de desenvolvimento e teste.

4.2.2.2 Componentes principais (*core components, add-ons*)

- **Core components:** Essas distribuições leves retêm os principais componentes do Kubernetes, como API Server, Scheduler, Controller Manager e etcd (ou uma alternativa leve).
- **Add-ons:** Podem incluir *add-ons* opcionais que fornecem funcionalidades adicionais, como *ingress controllers*, *dashboards* e suporte para diferentes tipos de armazenamento.



4.2.2.3 Exemplos de uso

- **Desenvolvimento local:** Desenvolvedores podem usar distribuições leves para criar *clusters* Kubernetes locais rapidamente, facilitando o teste e a integração contínua.
- **Ambientes de bordas:** k3s e MicroK8s são ideais para implantações em bordas de rede nas quais os recursos são limitados e a simplicidade é crucial.
- **Dispositivos IoT:** Essas distribuições permitem a orquestração de contêineres em dispositivos IoT, proporcionando gerenciamento centralizado e atualização simplificada.

4.2.2.4 Vantagens e desvantagens

Vantagens:

- **Leveza:** Requerem menos recursos de CPU e memória, tornando-as adequadas para ambientes com recursos limitados.
- **Facilidade de instalação:** São geralmente mais fáceis e rápidas de instalar e configurar.
- **Portabilidade:** Facilitam o desenvolvimento e o teste local, podendo ser facilmente movidos para *clusters* de produção completos.

Desvantagens:

- **Funcionalidades limitadas:** Podem não suportar todas as funcionalidades avançadas do Kubernetes padrão.
- **Menos suporte comercial:** Algumas distribuições podem ter menos suporte comercial ou comunidade menor comparada ao Kubernetes completo.
- **Escalabilidade:** Podem ser menos escaláveis e adequadas para grandes implantações em comparação com o Kubernetes padrão.



4.3 Ferramentas de automação

4.3.1 Terraform

O Terraform é uma ferramenta *open source* de Infraestrutura como Código (IaC) desenvolvida pela HashiCorp. Ele permite que os usuários definam e provisionem a infraestrutura de TI em várias plataformas de nuvem por meio de um código declarativo.

- **Arquitetura:** O Terraform utiliza uma linguagem de configuração própria (HCL – *HashiCorp Configuration Language*) para definir a infraestrutura desejada. Seus principais componentes incluem:
 - **Providers:** Plugins que interagem com as APIs dos provedores de nuvem para criar e gerenciar recursos.
 - **Resources:** Elementos da infraestrutura que são definidos no código (ex.: instâncias de computação e redes).
 - **Modules:** Conjuntos reutilizáveis de configurações que agrupam múltiplos recursos para facilitar a reutilização e a organização do código.

Exemplos de uso:

- **Implantação de infraestrutura em nuvem:** Utilizado para criar e gerenciar infraestrutura em provedores como AWS, Azure e GCP.
- **Gerenciamento de configuração:** Automatiza a configuração de rede, segurança e recursos computacionais.
- **Migração de infraestrutura:** Facilita a migração de ambientes de desenvolvimento para produção de forma consistente.

Vantagens

- **Multiprovedor:** Suporta uma ampla gama de provedores de nuvem e serviços, oferecendo flexibilidade e portabilidade.
- **Infraestrutura como código:** Permite a definição de infraestrutura de forma declarativa, facilitando a automação e a repetibilidade.
- **Comunidade e suporte:** Grande comunidade e ampla documentação, além de integração com outras ferramentas da HashiCorp.



Desvantagens:

- **Curva de aprendizado:** A linguagem HCL e o conceito de "estado" podem ser desafiadores para iniciantes.
- **Gerenciamento de estado:** O arquivo de estado do Terraform deve ser gerenciado com cuidado para evitar inconsistências e conflitos.
- **Complexidade em grandes ambientes:** Gerenciar grandes ambientes com muitos recursos interdependentes pode tornar-se complexo.

4.3.2 CrossPlane

O CrossPlane é uma plataforma de gerenciamento de infraestrutura *open source* que utiliza a API Kubernetes para provisionar e gerenciar recursos de nuvem de maneira declarativa. Permite que os operadores definam, configurem e gerenciem a infraestrutura de forma consistente em múltiplos provedores de nuvem.

A arquitetura do CrossPlane se baseia em:

- **Compositions:** Conjunto de recursos que define como um recurso de infraestrutura deve ser configurado e gerenciado.
- **Providers:** Interfaces para diferentes provedores de nuvem (AWS, Azure, GCP etc.) que permitem o gerenciamento de recursos específicos de cada provedor.
- **Workloads:** Definições de como as cargas de trabalho devem ser executadas e gerenciadas na infraestrutura provisionada.

Exemplos de uso:

- **Provisionamento multinuvm:** Gerencia recursos de infraestrutura de maneira consistente em vários provedores de nuvem.
- **Automação de infraestrutura como código (IaC):** Define infraestrutura usando práticas de IaC para facilitar a repetibilidade e a manutenção.
- **Gestão de serviços gerenciados:** Facilita a implementação e o gerenciamento de serviços gerenciados como bancos de dados e filas de mensagens.

Vantagens

- **Consistência:** Proporciona uma maneira unificada de gerenciar infraestrutura em múltiplos provedores de nuvem.



- **Flexibilidade:** Suporta uma ampla variedade de recursos e permite a criação de composições complexas.
- **Integração com Kubernetes:** Aproveita a API Kubernetes para gerenciar infraestrutura e cargas de trabalho de maneira declarativa.

Desvantagens:

- **Curva de aprendizado:** Pode ser complexo para novos usuários que não estão familiarizados com Kubernetes e práticas de IaC.
- **Maturidade:** Por ser uma tecnologia relativamente nova, pode ter menos suporte e menos casos de uso documentados comparado a ferramentas mais estabelecidas.
- **Dependência de Kubernetes:** Requer um *cluster* Kubernetes para operar, o que pode adicionar complexidade adicional à infraestrutura.

TEMA 5 – PROCESSAMENTO EM LOTE

5. 1 Introdução ao processamento em lote

1. **Definição de processamento em lote:** O processamento em lote é uma técnica de computação na qual os dados são coletados, processados e analisados em grandes quantidades de uma só vez, em vez de serem manipulados em tempo real. Essa abordagem é amplamente utilizada para tarefas que não exigem resposta imediata, como geração de relatórios, análise de grandes volumes de dados e tarefas de *back-end*.
2. **Importância do processamento em lote:** O processamento em lote é crucial para muitas operações empresariais e científicas. Ele permite a manipulação eficiente de grandes volumes de dados, otimiza o uso dos recursos computacionais e pode ser agendado para ocorrer durante períodos de baixa demanda de recursos, como à noite ou nos fins de semana.
3. **História e evolução do processamento em lote:** Historicamente, o processamento em lote surgiu com os primeiros computadores *mainframe*, nos quais tarefas eram processadas em lotes para maximizar o uso do tempo de processamento. Com a evolução da tecnologia, o processamento em lote foi adaptado para funcionar em ambientes de



computação em nuvem, oferecendo escalabilidade e flexibilidade aprimoradas.

5.2. Arquitetura de processamento em lote

O processamento em lote é uma técnica essencial na computação moderna, permitindo que grandes volumes de dados sejam processados de maneira eficiente e sistemática. Ao contrário do processamento em tempo real, no qual os dados são tratados à medida que chegam, o processamento em lote coleta e processa dados em intervalos definidos, geralmente fora do horário de pico ou durante janelas de manutenção.

A arquitetura de processamento em lote é crucial para maximizar a eficiência e a escalabilidade dos sistemas que lidam com grandes volumes de dados. Uma arquitetura bem projetada garante que as tarefas sejam executadas de forma ordenada, eficiente e com o mínimo de recursos necessários. Isso é particularmente importante em cenários como a geração de relatórios, análise de dados históricos, processamento de transações em massa e muitos outros.

5.2.1 Componentes de uma arquitetura de processamento em lote

- **Dados de entrada:** Dados coletados de várias fontes que serão processados.
- **Processadores de lote:** Sistemas que executam o processamento dos dados em lote.
- **Dados de saída:** Resultados gerados após o processamento dos dados.

5.2.2 Fluxo de trabalho em processamento em lote

Um fluxo de trabalho típico de processamento em lote inclui a coleta de dados, pré-processamento, processamento principal, pós-processamento e armazenamento ou exportação dos resultados.

5.2.3 Ferramentas e tecnologias comuns

Ferramentas como Apache Hadoop, Apache Spark e tecnologias de containers como Docker e Kubernetes são frequentemente usadas para implementar soluções de processamento em lote.



5.3. Serviços de processamento em lote em provedores de nuvem

5.3.1 AWS Batch

O AWS Batch é um serviço gerenciado que permite o fácil e eficiente processamento de trabalhos em lote de qualquer escala. Ele elimina a necessidade de gerenciar a infraestrutura subjacente, facilitando a execução de milhares de *jobs* em paralelo. AWS Batch provisiona dinamicamente a quantidade e tipo certo de recursos computacionais (como instâncias EC2 ou *Spot Instances*) com base no volume e requisitos específicos dos *jobs* submetidos.

- **Ambiente de computação:** Define os recursos de computação utilizados para executar os *jobs*. Pode incluir instâncias sob demanda ou *spot*.
- **Fila de *jobs*:** Armazena os *jobs* submetidos até que eles sejam executados. As filas ajudam a gerenciar a ordem e a prioridade de execução dos *jobs*.
- **Definição de *jobs*:** Especifica como os *jobs* serão executados, incluindo detalhes como a imagem do Docker, variáveis de ambiente e requisitos de recursos.

Quando um *job* é submetido, o AWS Batch verifica os requisitos de recursos e o programa na fila apropriada. Em seguida, ele provisiona os recursos necessários, executa o *job* e, após a conclusão, libera os recursos. Utilize o AWS Batch para reexecutar *jobs* falhos ou ajustar configurações conforme necessário.

5.3.2 Azure Batch

Azure Batch é um serviço de processamento de tarefas em lote que permite a execução de aplicações paralelas de alta escala e alto desempenho. Ele gerencia automaticamente a infraestrutura, distribuindo as tarefas entre múltiplas máquinas virtuais, escalando conforme necessário para atender às demandas das cargas de trabalho.

Azure Batch permite que os usuários definam trabalhos e tarefas a serem executados em um *cluster* de máquinas virtuais. Os componentes principais do Azure Batch são:



- **Pool de computação:** Conjunto de máquinas virtuais que executam as tarefas. Os *pools* podem ser escalados automaticamente com base nas necessidades do *job*.
- **Jobs e tarefas:** Um *job* é uma coleção de tarefas. Cada tarefa é uma unidade de trabalho que é executada em uma máquina virtual do *pool*.
- **Fila de jobs:** Os *jobs* são organizados em uma fila e atribuídos a VMs no *pool* conforme os recursos estão disponíveis.

Quando um *job* é submetido, Azure Batch distribui as tarefas pelas VMs do *pool*. Cada tarefa pode executar de forma independente e em paralelo, permitindo um processamento rápido e eficiente de grandes volumes de dados.

5.3.3 Google Cloud Dataflow

O Google Cloud Dataflow é um serviço gerenciado para processamento em lote. Ele é o componente principal para a construção de pipelines para coletar, transformar e realizar a saída de dados. O Google Cloud Dataflow utiliza a API do Apache Beam, que seria um modelo combinando fluxo de dados e processamento em lote. Ele não requer que haja configuração de instâncias de computação ou *clusters*, o que, em computação em nuvem, podemos chamar de *NoOps*. O Cloud Dataflow se integra com os serviços de Dados do Google Cloud Platform como Pub/Sub, BigQuery e toda a *stack* Cloud ML Engine.

5.4 Práticas de otimização de processamento em lote

- **Tuning de performance:** O processamento em lote é essencial para garantir que as tarefas sejam executadas de maneira eficiente e rápida. O *tuning* envolve a otimização de vários parâmetros e recursos para melhorar o desempenho geral do sistema.
- **Principais técnicas de tuning de performance:**
 - **Alocação de recursos:** Assegure-se de que cada *job* tenha a quantidade adequada de CPU, memória e armazenamento. Evite a sobrecarga de recursos para que as tarefas não os disputem.
 - **Paralelismo:** Utilize o paralelismo para dividir as tarefas em subtarefas menores que podem ser executadas simultaneamente. Ferramentas como Apache *Spark* e *Hadoop* facilitam essa divisão.



- **Balanceamento de carga:** Distribua a carga de trabalho uniformemente entre os recursos disponíveis para evitar gargalos. Balanceadores de carga ajudam a gerenciar essa distribuição.
- **Caching:** Use *caching* para armazenar dados intermediários, reduzindo a necessidade de recalcular ou reprocessar dados repetidamente.
- **Otimização de I/O:** Reduza o tempo de leitura e gravação de dados otimizando as operações de entrada e saída (I/O). Utilize discos SSD para armazenamento rápido e configure *buffers* adequados.

5.4.1 Gerenciamento de recursos

Crucial para maximizar a eficiência do processamento em lote e garantir que os recursos sejam utilizados de maneira otimizada.

5.4.2 Estratégias de gerenciamento de recursos:

- **Alocação dinâmica:** Use técnicas de alocação dinâmica de recursos para ajustar a quantidade de CPU, memória e armazenamento de acordo com as necessidades do *job*.
- **Escalonamento de tarefas:** Planeje e agende tarefas de maneira eficiente para garantir que os recursos sejam utilizados da melhor forma possível. Ferramentas de orquestração como Kubernetes podem ajudar nesse processo.
- **Monitoramento contínuo:** Monitore continuamente o uso de recursos para identificar e resolver rapidamente quaisquer ineficiências ou problemas.
- **Ajustes automáticos:** Implemente políticas de ajuste automático que possam aumentar ou diminuir a alocação de recursos com base na carga de trabalho atual.

5.4.3 Monitoramento e *logging*

Monitoramento e *logging* são fundamentais para manter a visibilidade e o controle sobre o processamento em lote. Eles ajudam a identificar problemas, otimizar o desempenho e garantir que os *jobs* sejam executados conforme planejado.



5.4.4 Práticas de monitoramento e *logging*

- **Monitoramento em tempo real:** Utilize ferramentas como AWS CloudWatch, Azure Monitor ou Google Cloud Monitoring para acompanhar o desempenho dos *jobs* em tempo real.
- **Logs detalhados:** Configure *logging* detalhado para capturar todas as atividades e eventos durante o processamento. Isso facilita a identificação de erros e gargalos.
- **Alertas e notificações:** Configure alertas para notificar a equipe de operações sobre quaisquer problemas ou anomalias que ocorram durante o processamento.
- **Análise de logs:** Use ferramentas de análise de *logs* para extrair *insights* sobre o desempenho do sistema e identificar áreas para melhoria.

5.5 Desafios e considerações no processamento em lote

5.5.1 Gerenciamento de erros

O gerenciamento de erros no processamento em lote é essencial para garantir que falhas sejam tratadas de maneira eficaz, minimizando o impacto no processamento global.

Estratégias de gerenciamento de erros:

- **Deteção e registro de erros:** Implemente mecanismos robustos para detectar e registrar erros. *Logs* detalhados ajudam a entender a causa raiz das falhas.
- **Retry policies:** Configure políticas de reexecução automática para tarefas que falham devido a problemas temporários, como falhas de rede.
- **Isolamento de falhas:** Isolar falhas para que uma tarefa problemática não afete o restante do *job*. Use contêineres ou VMs para isolar processos.
- **Alertas proativos:** Configure alertas para notificar os administradores sobre erros críticos, permitindo uma resposta rápida.



5.5.2 Escalabilidade e elasticidade

Escalabilidade e elasticidade são características chave para um sistema de processamento em lote eficiente, permitindo que o sistema cresça e se adapte conforme necessário.

Práticas para melhorar a escalabilidade e elasticidade:

- **Escalabilidade horizontal:** Adicione mais instâncias de computação para lidar com aumentos de carga, distribuindo o trabalho entre várias máquinas.
- **Escalabilidade vertical:** Aumente os recursos de instâncias existentes (CPU, memória) para lidar com cargas de trabalho mais intensivas.
- **Auto scaling:** Utilize recursos de *auto scaling* oferecidos pelos provedores de nuvem para ajustar automaticamente a capacidade de computação com base na demanda.
- **Arquitetura sem estado:** Desenvolva aplicações sem estado para facilitar a distribuição e a escalabilidade, em que o estado é mantido em um repositório externo como um banco de dados.

5.5.3 Custos e eficiência

Custos e eficiência são considerações críticas no gerenciamento de processamento em lote, cujo objetivo é maximizar a eficiência enquanto se mantém os custos sob controle.

Estratégias para gerenciar custos e eficiência:

- **Otimização de recursos:** Utilize ferramentas de monitoramento para identificar recursos subutilizados e ajustar a alocação conforme necessário.
- **Instâncias *spot* e reserva:** Utilize instâncias de baixo custo como Spot Instances (AWS) ou Preemptible VMs (GCP) para economizar em tarefas não críticas.
- **Planejamento de capacidade:** Preveja a demanda e planeje a capacidade de recursos com antecedência para evitar custos inesperados.



- **Economia de escala:** Aproveite descontos por volume e outras economias de escala oferecidas pelos provedores de nuvem para grandes volumes de processamento.

FINALIZANDO

O gerenciamento de máquinas virtuais e contêineres é crucial na infraestrutura de TI moderna. Ferramentas de orquestração como Kubernetes, distribuições leves de Kubernetes, CrossPlane e Terraform são fundamentais ao permitir que as organizações automatizem, escalem e mantenham suas aplicações de maneira eficiente e resiliente. Kubernetes, com sua robustez e ampla adoção, continua sendo a escolha principal para a orquestração de contêineres, enquanto as distribuições leves, como k3s e MicroK8s, oferecem alternativas simplificadas para ambientes com recursos limitados.

Por outro lado, CrossPlane e Terraform fornecem abordagens poderosas para o provisionamento de infraestrutura como código, permitindo que os operadores gerenciem recursos de nuvem de maneira declarativa e consistente. Cada ferramenta tem seus pontos fortes e fracos, e a escolha da ferramenta correta dependerá das necessidades específicas de cada organização, da complexidade da infraestrutura e das habilidades técnicas da equipe.

Adotar essas ferramentas melhora a eficiência operacional, garantindo que as organizações possam responder rapidamente às mudanças no ambiente de negócios, mantendo a flexibilidade e a escalabilidade necessárias para crescer e inovar. À medida que a tecnologia continua a evoluir, essas ferramentas também se adaptarão, oferecendo novas funcionalidades e abordagens para enfrentar os desafios futuros do gerenciamento de infraestrutura.



REFERÊNCIAS

AWS Certified Solutions Architect – Associate. **AWS**, 2024 Disponível em: <https://aws.amazon.com/certification/certified-solutions-architect-associate/>.

Acesso em: 24 out. 2024.

BURNS, B.; BEDA, J.; HIGHTOWER, K. **Kubernetes**: Up & Running. Newton: O'Reilly Media, 2019.

CROSSPLANE. Disponível em: <https://crossplane.io>. Acesso em: 24 out. 2024.

HASHICORP: TERRAFORM COMMUNITY. Disponível em: <https://www.terraform.io>. Acesso em: 24 out. 2024.

HIGHTOWER, K.; BURNS, B.; BEDA, J. **Kubernetes**: Up and Running: Dive into the Future of Infrastructure. Newton: O'Reilly Media, 2017.

KUBERNETES. Disponível em: <https://kubernetes.io> Acesso em: 24 out. 2024.

LIGHTWEIGHT Kubernetes. **Rancher Labs**, 2024. Disponível em: <https://k3s.io>. Acesso em: 24 out. 2024.

MICROSOFT Certified: Azure Solutions Architect Expert. **Microsoft**, 2024. Disponível em: <https://learn.microsoft.com/en-us/certifications/azure-solutions-architect/>. Acesso em: 24 out. 2024.

PROFESSIONAL Data Engineer Exam Guide. **Google Cloud**, 2024. Disponível em: <https://cloud.google.com/certification/guides/data-engineer>. Acesso em: 24 out. 2024.

THE EFFORTLESS Kubernetes. **MicroK8s**, 2024. Disponível em: <https://microk8s.io>. Acesso em: 24 out. 2024.

THE LINUX FOUNDATION. **The Kubernetes Ecosystem**. Disponível em: <https://www.linuxfoundation.org>. Acesso em: 24 out. 2024.